



Misurazioni prestazionali tramite PERF su software MPI

Relazione Attività Progettuale

Sistemi Operativi M

Anno 2024/25

Studente: **Nicolò Venieri**

Professoressa: **Anna Ciampolini**

Indice

Capitolo 1: L’Infrastruttura HW & SW	3
1.1 Obbiettivi e Strumenti	4
1.2 Architettura Hardware.....	4
1.3 Sistemi per il profiling energetico	4
1.3.1 Rapl	4
1.3.2 Perf	5
1.3.3 Powercup	5
Capitolo 2: Implementazioni	6
2.1 Sbatch con comando Perf-Stat	6
2.2 Letture con Syscall su MPI	6
2.3 Libreria comandi utili	8
Capitolo 3: Dati osservati e Conclusioni	8
3.1 Osservazioni sui dati ottenuti.....	8
3.2 Conclusioni.....	9

Capitolo 1: L'Infrastruttura HW & SW

1.1 Obiettivi e Strumenti

Lo scopo di questo studio è analizzare la possibilità di misurare i consumi energetici di codice MPI su un'infrastruttura HPC, utilizzando diversi strumenti messi a disposizione dall'ambiente Linux, eventualmente integrati con implementazioni in linguaggio C inserite all'interno di codice MPI.

1.2 Architettura Hardware

Leonardo si articola in due sezioni principali: il **Booster Module**, sul quale ci appoggeremo per sviluppare questa attività progettuale, orientato al calcolo massivamente parallelo e il **Data-Centric Module**, progettato per supportare elaborazioni di tipo più tradizionale e CPU-intensive.

- **Booster Module:** è basato su blade server **BullSequana X2135 “Da Vinci”**. Il sistema comprende circa **3 456 nodi**, ciascuno con:
 - **CPU:** 1× **Intel Xeon Platinum 8358 (Ice Lake)**, 32 core, 2.6 GHz
 - **GPU:** 4× NVIDIA Ampere A100 da 64 GB HBM2e, con NVLink 3.0 (200 GB/s)
 - **RAM:** 512 GB DDR4 3200 MHz per nodo
 - **Interconnessione:** 2× dual-port HDR100 InfiniBand (400 Gbps per nodo)
- **Data-Centric (General Purpose) Module:** è composto da **1 536 nodi**, ciascuno basato su:
 - **Blade:** BullSequana X2140 (o X2610 in alcune fonti)
 - **CPU:** 2× **Intel Sapphire Rapids**, 56 core ciascuna (totale 112 core per nodo), con frequenze intorno a 2.0 GHz o fino a 4.8 GHz in alcune configurazioni
 - **RAM:** 512 GB (16×32 GB) DDR5 4800 MHz per nodo
 - **Interconnessione:** 1× 100 Gbps NVIDIA HDR InfiniBand per nodo

1.3 Sistemi per il profiling energetico

1.3.1 Rapl

RAPL (Running Average Power Limit) è un'interfaccia a basso livello progettata inizialmente da Intel per la gestione dell'energia, che consente di **misurare il consumo energetico** e la potenza assorbita da varie parti di un calcolatore, chiamate **domini**. Questa tecnologia misura il consumo di elettricità della CPU e di altri componenti, come la memoria DRAM, attraverso sensori integrati nel system-on-chip. Tali misurazioni sono rese disponibili al sistema operativo attraverso i **registri specifici del modello (MSR - Model-Specific Registers)**, che sono registri a basso livello forniti

dalla CPU stessa. Ogni dominio RAPL possiede un contatore di energia specifico, accessibile attraverso registri come MSR_d_ENERGY_STATUS. Il principale problema tecnico nell'uso di RAPL è l'**overflow dei contatori di energia** MSR. Questi contatori hanno una dimensione limitata e possono facilmente andare in overflow, il che può avvenire dopo circa **"60 secondi sotto carico pesante"**. Poiché la CPU non segnala direttamente l'overflow, l'unico modo per garantire l'accuratezza della misurazione è eseguire il **polling dei valori con una frequenza sufficiente** e applicare una correzione specifica per l'overflow, calcolando la differenza di energia (Δm) tra due letture successive.

1.3.2 Perf

Perf è un'interfaccia del kernel Linux che fornisce capacità di monitoraggio delle prestazioni orientate agli eventi. È uno dei due meccanismi ad alto livello offerti da Linux per interrogare i registri del processore forniti da RAPL. perf gestisce due tipi di eventi: **eventi di conteggio**, il cui valore più recente deve essere interrogato periodicamente, ed **eventi di campionamento**, che vengono aggiunti periodicamente a un buffer. I contatori di consumo energetico RAPL sono stati integrati come **eventi di conteggio**. A differenza di tecnologie come powercup perf è **quasi immune agli overflow**. Sebbene l'overflow sia teoricamente possibile (essendo un intero a 64 bit), il sottosistema corregge automaticamente questi eventi, rendendo la probabilità di errori molto più bassa. L'elenco degli eventi disponibili può essere letto dal sysfs, ispezionando la directory `/sys/devices/power/events/` Per accedere alle misurazioni del consumo energetico, è necessario aprire l'evento tramite la syscall `perf_event_open`. Il descrittore di file ottenuto viene poi letto periodicamente. È fondamentale aprire ogni evento **esattamente una volta per ogni socket** della CPU.

Per l'esecuzione, perf-events richiede alcune **privilegi**: la *capability* **CAP_PERFMON** (introdotta in Linux 5.8; per i kernel più vecchi è richiesta `CAP_SYS_ADMIN`) o che l'impostazione del kernel `perf_event_paranoid` sia impostata su 0 o -1. Sebbene sia considerato **più difficile da usare negli script Bash** rispetto a powercap (a causa della chiamata a `perf_event_open`), in un ambiente compatibile, perf-events è **altrettanto facile da mantenere** nel tempo quanto powercap e può adattarsi facilmente per supportare nuovi domini o microarchitetture.

1.3.3 Powercup

Il **Power Capping Framework** è un'interfaccia software di alto livello fornita dal **kernel Linux** che si posiziona sopra l'interfaccia RAPL. Esso permette di controllare RAPL e di accedere alle misurazioni dallo spazio utente (userspace) tramite l'uso del **file system virtuale sysfs**. Non richiede l'uso di specifiche capability di Linux (come `CAP_PERFMON`), necessitando solo dell'accesso in lettura alla directory `intel-rapl`. La principale difficoltà con powercap è che gli **overflow si verificano frequentemente**,

rendendo necessaria l'applicazione della correzione. Inoltre, il meccanismo , secondo gli studi riportati in bibliografia, ha mostrato una **latenza di lettura maggiore** rispetto a perf-events

Capitolo 2: Implementazioni

2.1 Sbatch con comando Perf-Stat

Ho inserito inizialmente all'interno del codice Sbatch una chiamata **perf stat** , dove ho specificato il package e indicato il processo (srun ./processompi) del quale tenere traccia dei consumi. A questo comando possono essere susseguiti vari parametri per suddividere l'insieme di dati dei quali tener conto.

Il parametro **-e power/energy-pkg/** permette di ottenere la lettura di un determinato evento. In questo caso il consumo di energia.

Senza ulteriori parametri perf calcola il completo del consumo di tutta **la /le** CPU, quindi tutti i cores unificati. Nel nostro caso non è un dato accurato in quanto questa è una lettura influenzata da Rumore.

Durante l'intervallo di tempo ΔT in cui perf misura il consumo energetico, l'attività di altri processi in esecuzione sul processore può andare ad influire sul consumo energetico. Per questo motivo il valore ottenuto può risultare meno preciso.

2.2 Letture con Syscall su MPI

Per ottenere un livello di dettaglio maggiore rispetto all'uso del comando da riga di comando perf stat, è stata implementata una lettura tramite la **system call perf_event_open**. Questa syscall consente di aprire e configurare manualmente un evento del kernel o dell'hardware (tramite i Performance Monitoring Counters – PMC) e di leggere il suo valore direttamente dal programma in esecuzione.

Nel caso specifico, il codice MPI sviluppato utilizza perf_event_open per accedere ai contatori **RAPL (Running Average Power Limit)** esposti dalle CPU Intel. Questi contatori permettono di misurare il consumo energetico del package CPU.

L'approccio seguito è il seguente:

Configurazione dell'evento: si definisce una struttura perf_event_attr in cui vengono specificati tipo e codice dell'evento da monitorare. In questo caso, l'evento configurato è legato al consumo energetico (lettura RAPL).

```
// Set up the perf_event_attr struct
struct perf_event_attr attr;
memset(&attr, 0, sizeof(attr));
attr.type = type;
attr.size = sizeof(struct perf_event_attr);
attr.config = config;
```

Apertura del contatore: tramite `syscall(PERF_EVENT_OPEN, &attr, ...)` si ottiene un file descriptor che rappresenta l'evento aperto

```
// Open the event
long fd = syscall(PERF_EVENT_OPEN_SYSCALL_NR, &attr, -1, 0, -1, 0);
if (fd == -1) {
    perror("perf_event_open");
    return 1;
```

Lettura del valore iniziale: usando `read(fd, &counter_before, sizeof(counter_before))` si salva lo stato del contatore prima dell'esecuzione del calcolo MPI.

```
// Read initial counter value
long int counter_before;
if (read(fd, &counter_before, sizeof(long int)) != sizeof(long int)) {
    perror("read");
    close(fd);
    return 1;
}
```

Esecuzione del calcolo MPI: i processi si scambiano i vettori, li sommano e calcolano le somme parziali.

Lettura del valore finale: al termine del calcolo, una nuova lettura (`read(fd, &counter_after, ...)`) permette di ottenere lo stato finale del contatore.

Calcolo dell'energia consumata: la differenza tra i due valori, opportunamente moltiplicata per un fattore di scala fornito dal kernel, fornisce l'energia in Joule consumata durante l'elaborazione.

```
// Read the final counter value
long int counter_after;
if (read(fd, &counter_after, sizeof(long int)) != sizeof(long int)) {
    perror("read");
    close(fd);
    return 1;
}

// Calculate energy usage
double scale = 2.3283064365386962890625e-10; // from energy-cores.scale
double energy = (counter_after - counter_before) * scale;
printf("Energy usage: %.100f Joules\n", energy);

// Close the event
close(fd);
return 0;
```

Tuttavia, questa integrazione considera solamente due rank divisi su un job MPI, e conseguentemente un'implementazione su codice che richiede tanti rank può generare Overhead

2.3 Libreria comandi utili

Conseguentemente ai problemi individuati nelle due sezioni precedenti ho pensato di integrare all'interno di alcune librerie delle possibili funzioni utili per reperire all'interno di codice C, informazioni sulla CPU nel quale il software sta eseguendo.

```
// Inizializzazione manuale (con socket specificato)
int energy_meter_init_manual(energy_meter_t* m, int socket, energy_zone_t zone);

// Inizializzazione automatica (detect socket corrente)
int energy_meter_init(energy_meter_t* m, energy_zone_t zone);

// Avvio e stop della misurazione
int energy_meter_start(energy_meter_t* m);
int energy_meter_stop(energy_meter_t* m);

// Calcolo dei joule
double energy_meter_joules(energy_meter_t* m);

// Utility: ottiene il socket corrente (dal core fisico in esecuzione)
int get_current_socket();
```

Capitolo 3: Dati osservati e Conclusioni

3.1 Osservazioni sui dati ottenuti

1. Verifica preliminare dell'emissione dei dati

In una prima fase, è stata effettuata una verifica sulla corretta emissione dei dati. Lanciando una misurazione con **perf** sul compilato **MPI**, veniva generato in output un valore iniziale. Il passo successivo è stato quindi quello di rilanciare un job identico, con l'obiettivo di ottenere una seconda misurazione utile al confronto e alla validazione del dato.

2. Acquisizione di due misurazioni confrontabili

A seguito di questa prima analisi, l'attenzione si è concentrata sull'ottenimento di due misurazioni che potessero essere confrontate in modo accurato.

A tal fine, è stata eseguita una misurazione con **perf** sul compilato MPI e, contemporaneamente, una misurazione interna al software stesso, sfruttando le librerie e le **syscall perf** per rilevare l'attività della CPU sulla quale veniva

eseguito il processo. Ci siamo resi conto di aver usato un indirizzo sbagliato perché la lettura restituiva **0.0000** come valore energetico.

Grazie al parametro **verbose** di perf , abbiamo capito che l'indirizzo corretto era HEX93

3. Identificazione della CPU di esecuzione

L'ultimo tentativo è stato volto a determinare la **CPU di provenienza** del processo. Attraverso l'utilizzo della funzione sched.getCPU(), è stato possibile identificare la CPU sulla quale ogni rank MPI stava effettivamente lavorando.

Questa informazione si è rivelata particolarmente utile in vista di future implementazioni, in cui potrebbero essere presenti più misurazioni simultanee sulla stessa CPU.

3.2 Conclusioni

La prima osservaz

Per ottenere una misurazione accurata del consumo energetico di un processo MPI è necessario lanciare il JOB sbatch in modalità Exclusive, dunque richiedendo il nodo (o i nodi) in modalità completamente esclusiva. Questo parametro fa sì che SLURM assegni una priorità bassissima al processo causando tempi di attesa dell'esecuzione elevatissimi. In questo caso, l'approccio più fattibile è quello che coinvolge la coesistenza di altri processi, il che porta tuttavia a introdurre anche fattori di rumore e rendere quindi la misurazione non accurata. Il tentativo di identificare la CPU sulla quale il software veniva effettivamente eseguito è andato a buon fine, fornendo così uno strumento utile per il raggruppamento e l'analisi delle misurazioni. Una **futura attività** potrebbe essere il **raggruppamento dei dati** ottenuti divisi per cpu di "provenienza"

In conclusione, dalle analisi condotte è emerso che **PERF non è uno strumento accurato per la misurazione del consumo di codice sui sistemi CINECA**. Le letture risultano infatti fortemente influenzate dal rumore dovuto alla condivisione delle risorse di calcolo e alla difficoltà di sincronizzare correttamente le misurazioni. Anche i tentativi di misurazione interna ai rank non hanno prodotto dati sufficientemente stabili o attendibili. Perf risulta utile ad avere un'idea generica del consumo della CPU sulla quale avviene l'esecuzione del rank/processo.

Bibliografia

- <https://man7.org/linux/man-pages/man1/perf-stat.1.html>
- hpc.cineca.it Wikipedia The Next Platform Leonardo Pre-exascale Supercomputer
- https://man7.org/linux/man-pages/man3/sched_getcpu.3.html
- <https://achanda.dev/posts/measuring-energy-usage-c>
- <https://github.com/tud-zih-energy/lo2s/issues/128>
- <https://marcelgarus.dev/jupyter-energy>
- <https://man.archlinux.org/man/perf-list.1.en>